F.I.C. - curs

01.10.19.

CURS 1

Cap. 1: Mașina IAS și structura unui calculator

ENIAC – Electronic Numerical Integrate And Calculate

5.06.1943 US Army

University of Pennsylvania, Moore Sc. of EE

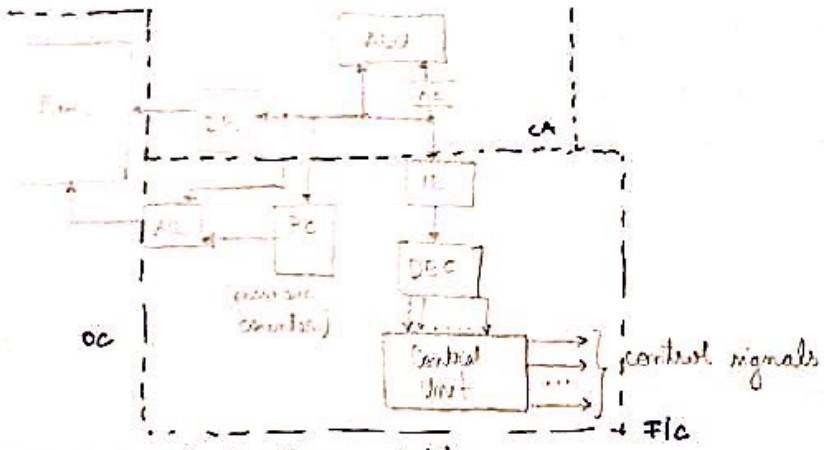
programs hard coded (switches, cables etc.)

Succesor: EDVAC 1946

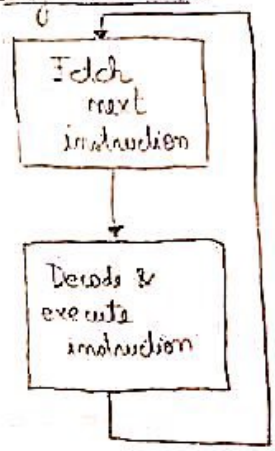
J. von Neumann → notes

→ IAS (University of Princeton)

1.1. Mașina IAS



Prog execution:



Memoria → unificată. (prog + date)

→ separată. (prog și date separat)

Set de instrucțiuni:

- Load x : AC ← Mem[x] (aduc din memorie)
- Store x : Mem[x] → DR (depozitez în memorie)
- Add x (op. aritmetice) : AC ← AC + Mem[x]
- And x (op. logice) : AC ← AC and Mem[x]
- Jump x : PC ← x
- JmpZ (x) : if (AC=0) then PC ← x

Scanned with CamScanner

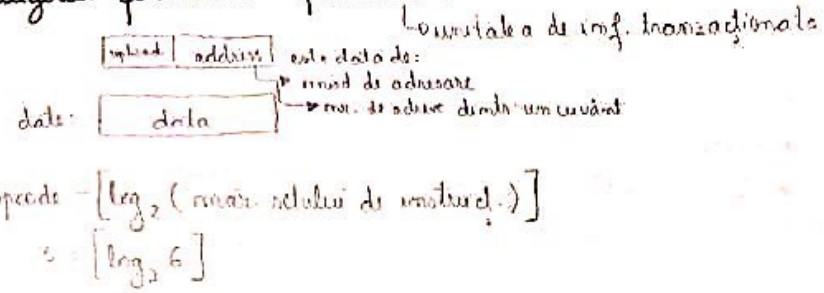
if reg. then
else ...

Pașii necesari construirii unei mașini de calcul :
1. Instrucțiuni
2. Tehnologii
3. Implementare

①. Instrucțiuni

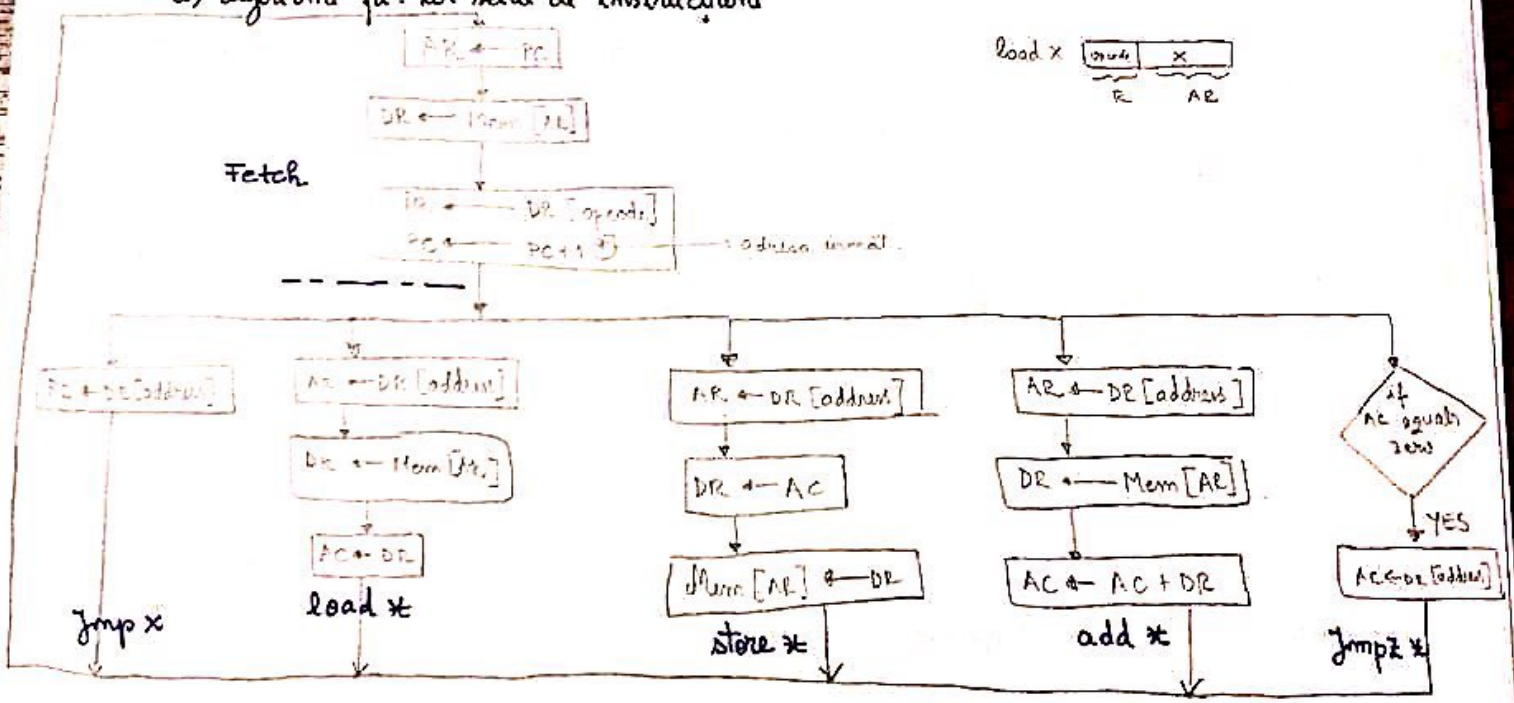
a) alegerea setului de instrucțiuni

b) alegerea formatului pt. cuvânt



c) stabilirea nr. de porturi I/O și dimensiunea memoriei.

d) algoritmi pt. tot setul de instrucțiuni



e) divizarea semnalelor de control

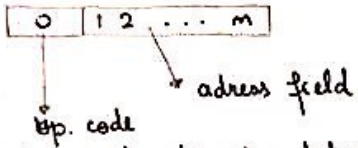
②. Tehnologii – alegerea tehnologiilor pentru implementare

Scanned with CamScanner

CURS 2 : 11.10.19.

1.2. Setul de instrucțiuni

Instruction word



m-operanzi, $m \le m$ expliciți, restul implicați.

Setul de instrucțiuni – determinat de operațiile care se doresc implementate.

Instrucțiunea e compusă din opcode și câmp de adresă.

$instr = f(x_1, x_2, \dots x_m)$

Câmpul de adresă poate să cont. 1 sau mai mulți operanzi, dați implicit | explicit.

Nr. de op. dați explicit rep. date sau adrese. Ceilalți rep. regiștri sau porturi.

Pt. un set de instruct., val. max. a unui m mic dă denumirea de: mașină cu m adrese (m-adress machine).

Orice instruct. e o fct. de operanzi numiți $x_1, x_2, \dots x_m$ (și cei implicite → vor fi hard-codați de op. code).

1.3. Moduri de adresare

↳ depind de cât de flexibilă și performantă e mașina de calcul

① Adresarea imediată: – nu oferă flexibilitate

– amândă câmpuri fixe, nu pot fi modificate

Load i 150

opcode	150

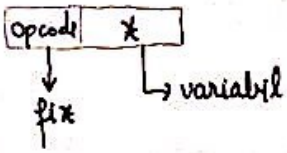
0 acces la memorie

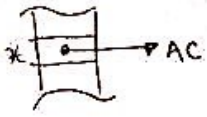
② Adresare directă:

- load i 150

AC ← 150

- load x ⟺ AC ← Mem[x] 1 acces la memorie

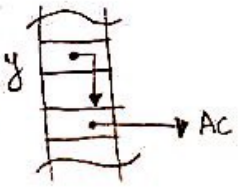


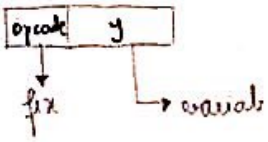


③ Adresare indirectă:

load In y ⟺ AC ← Mem[Mem[y]]

2 acces la memorie





Memoria consumă – timp, încetinește procesele.

– energie

mai mult ca procesorul

Scanned with CamScanner

1.4. Tipuri de adresare

- absolută – adresa dată în mod explicit pe întreg câmpul de adresă, sub forma unei valori
- relativă – partiționarea câmpului de adresă în: { a – adresa relativa; b – deplasamentul (bias) }

$f(A, B)$

	A	B

- adresa absolută = funcție între ad. rel. și deplasament
- deplasamentul – poate să nu conțină spațiile, dat implicit.

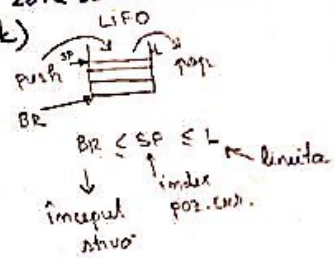
BR – base register → util pt. relocarea unor zone de memorie
IR – index register → lucrul cu stiva (stack)

↳ vectori

X [10]

X [0], X [1], X [2] ....

↑ adr. început



* stiva – transmiterea de parametrii
– apel de subrutine (întreruperi)

1.5 Secvențierea instrucțiunilor

(a). Cu adresă explicită

instr.	adresă	adresă

– se dublează necesarul de memorie (scump, nu merge rapid)
– avantaj: nu mai e nevoie de instrucț. de jump

(b). Adresare implicită: pe baza PC care în orice mom. conține adresa următ. instrucțiuni

Consecințe:
- PC trebuie incrementat
- apelul de subrutine se desf. în 2 pași:

1. salvarea contextului în stivă
2. încărcarea PC cu adresa de început a subrutinei

Scanned with CamScanner

15.10.19

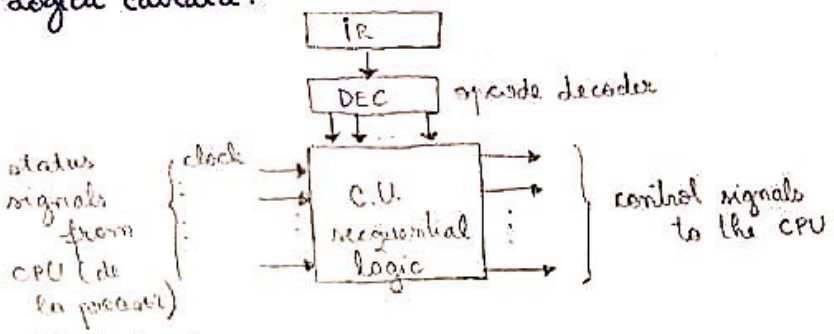
CURS 3

CAP. II: Design-ul unităților de control

2.1. Începuturile μP. Proiectul lui Wilkes.

. Design U.C. → logică cablată (hardwired logic)
→ logică microprogramată (preprogrammed logic)

A. Logica cablată:

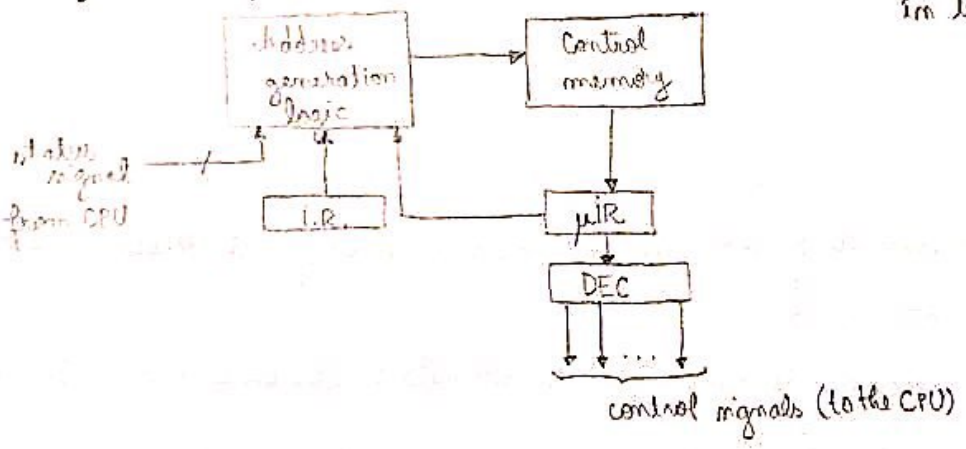


Metode
- state table
- delay element
- sequence counter
- one hot

, aproape echivalente.

B. Logica microprogramată

* secvența de la A se execută în buclă.



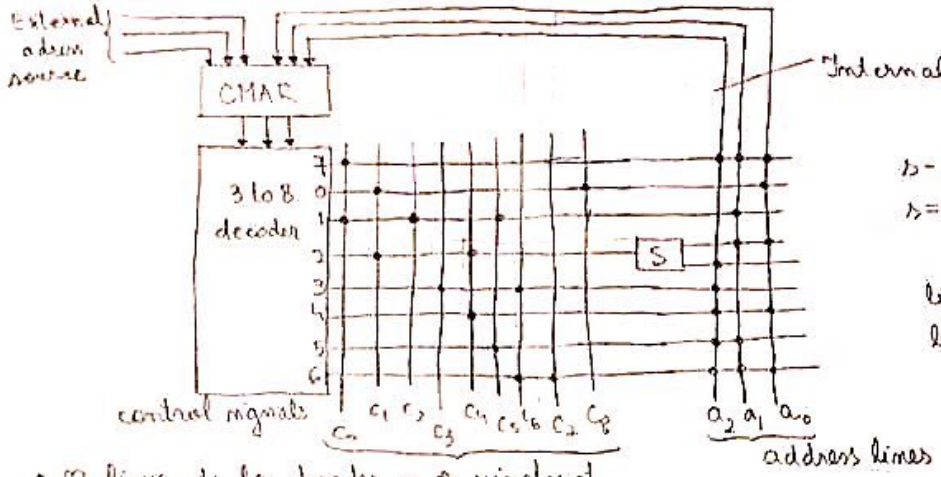
↳ IR prin AGL - gen. exec unei secv. de μinstr. care prin decodif. contribuie la gen. adreselor următ. de μinstrucț.

Tehnica microprog. e mai ușor de implem. decât cea cablată. ∃ posib. optimizării UC.

↳ aduce o flexib. suplimentară - memoria de control poate fi rescrisă.

1951: M. Wilkes - alternativă la logica cablată.

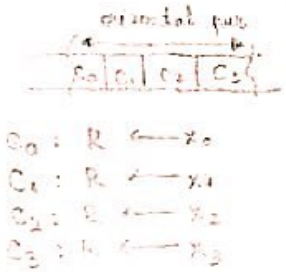
Scanned with CamScanner

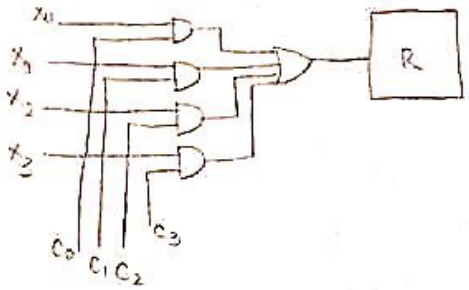


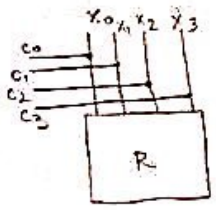
s - switch
s = 0 - prima linie (ad. linia 3)
1 - a 2a linie (ad. linia 4)
linia 6 → linia 7 etc...
linia 7 → linia 7 (terminarea unui algoritm)

- 8 linii de la decoder = 8 minstruct
- 8 minstruct. 2 categ de semnale — control / liniile de address

2.2. Formate de minstrucțiuni







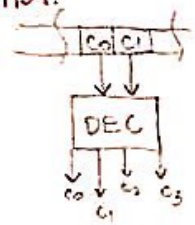
↳ reducerea dimensiunii pe verticală ⟺ adaos de decodif. pt. decodif. câmpurilor corup. semnalelor de control

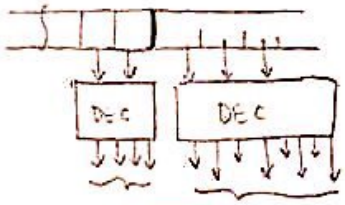
↳ nu pot fi 2 semnale de control active simultan (ar încărca o comb. log. între x-uri) — mutual exclusivitate.

↳ economisirea ⇒ codificare

↳ 4 semnale de control codif. pe 2 biți — se restrânge spațiul / se elimină competiția între semnale

CURS4:



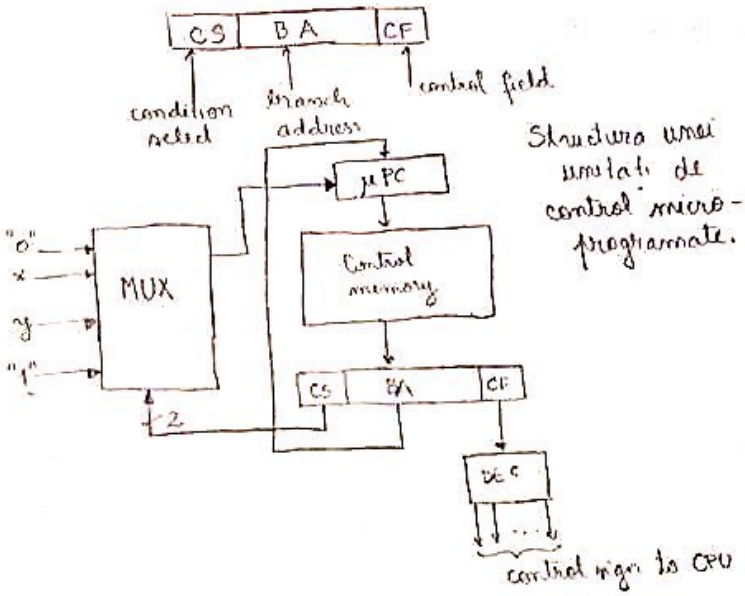


semnale de control mutual exclusive

Scanned with CamScanner

2.3. Adresarea microinstrucțiunilor

2.4. Organizarea unui UC μP



Structura unei unități de control microprogramate.

CS	
00	no branching
01	branch if x
10	branch if y
11	unconditional branching

2.5. Unit. de control microprogramată pentru algoritmul lui Robertson (pt. numere fracționare)

-8 16

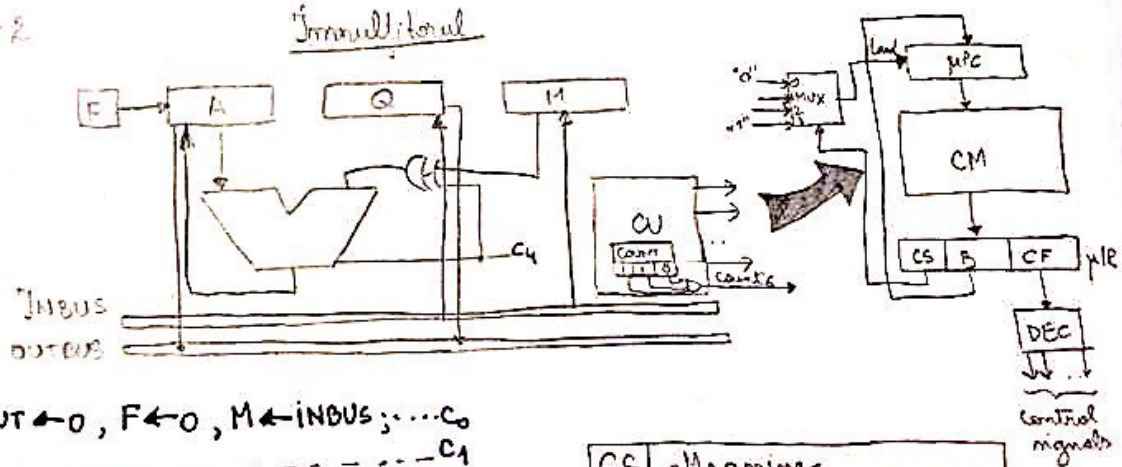
1 1 1 0 = -2

(C2)

1

1 1 0 1

0 0 1 0



0000 Begin: A ← 0, COUNT ← 0, F ← 0, M ← INBUS; ... C_0

0001 Q ← INBUS; ..... C_1

0010 Test1: if Q[0] = 0 go to RShift

0011 A ← A+M, F ← F + Q[7]M[7]. ..... C_2

0100 RShift: A.Q ← F.A.Q[7:1]; ..... C_3

0101 if COUNT6 = 1 go to Test2;

0110 COUNT ← COUNT + 1, go to Test1; ..... C_4

0111 Test2: if Q[0] = 0 go to Output;

1000 A ← A-M, Q[0] ← 0; ..... C_5, C_5

1001 Output: Outbus ← A; ..... C_6

1010 Outbus ← Q; ..... C_7

1011 End: go to End; ..... End

	CS	Meaning
0	00	no branching
1	01	branch if Q[0]=0
2	10	branch if COUNT6=1
3	11	uncond. branching

	Control signal	Meaning
1	C_0	F ← 0, A ← 0, COUNT ← 0, M ← inbus
2	C_1	Q ← INBUS
3	C_2	A ← A+M, F ← F + Q[0]·M[7]
4	C_3	Rshift F.A.Q.
5	C_4	count ← count+1
6	C_5	comp(M)
7	C_6	Outbus ← A
8	C_7	Outbus ← Q
9	End	end of current op

Scanned with CamScanner

CURS 5

- purely horizontal format – alocați direct, fără codif.
- branch adress (BA) or control field (CF)
- encoding by function

2.5.1. Formatul pur orizontal

<2>	<4>	<9>
CS	BA	CF

Cap CM = 12 instr. × 15 biți = 180 biți

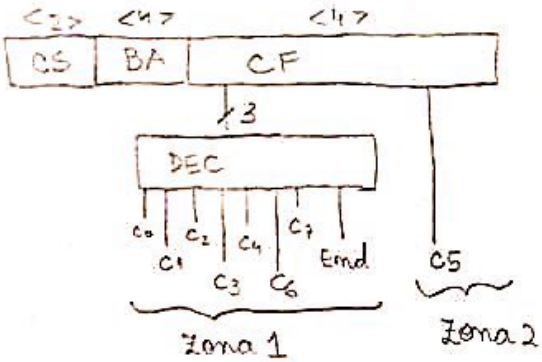
	Address în CM	CS	BA	CF: C_0 C_1 C_2 C_3 C_4 C_5 C_6 C_7 End
Begin	0000	00	0000	1 0 0 0 0 0 0 0 0
	0001	00	0000	0 1 0 0 0 0 0 0 0
Test 1	0010	01	0100	0 0 0 0 0 0 0 0 0
	0011	00	0000	0 0 1 0 0 0 0 0 0
RShift	0100	00	0000	0 0 0 1 0 0 0 0 0
	0101	10	0111	0 0 0 0 0 0 0 0 0
	0110	11	0010	0 0 0 0 1 0 0 0 0
Test 2	0111	01	1001	0 0 0 0 0 0 0 0 0
	1000	00	0000	0 0 1 0 0 1 0 0 0
Output	1001	00	0000	0 0 0 0 0 0 1 0 0
	1010	00	0000	0 0 0 0 0 0 0 0 1 0
	1011	11	1011	0 0 0 0 0 0 0 0 0 1

→ are dezavantajul de a risipi spațiul de mem., evidentă la instruct. no-branch. (se pierde câmpul de adr.) cât și prin câmpul de control (semnale de cont. inactive).

Se dorește a fi limitată prin codificarea câmpului de control.

Situația ideală presupune semnale de control mutual exclusive (nu sunt activate simultan), în realitate se întâmplă frecv. situații precum la adresa 1000, unde 2 semnale de control sunt active simultan, ceea ce înseamnă necesitatea partiț. CF în cel puțin 2 zone, cărora le vor corespunde semnalele de control C2, C5.

2.5.2. Formate cu codificarea CF. (Encoding by function)



Capacitatea CM = 12 instr. × 10 biți = 120 biți (codif. maximală)
(33% economie)

Encoding by function corelare funcțională a unor semnale

Inbus	
00	NO OPERATION
01	C_0
10	C_1
11	unused

Outbus	
00	NO OP.
01	C_6
10	C_7
11	End

RShift	
0	NO OP
1	C_3

Cont	
0	NO OP.
1	C_4

Scanned with CamScanner

Adder	
00	no op.
01	C_2
10	C_2, C_5
11	unused

} Total 8 biți CF
↓
2+2+1+1+2

Cap. CM = 12 x 14 biți = 168 biți.

→ C_2, C_5
Add Compl

CF
INBUS OUTBUS RSHIFT COUNT ADD COMPL
<2> <2> <1> <1> <1> <1>

Address in CM	CS	BA	CF					
			INBUS	OUTBUS	RSHIFT	COUNT	ADD	COMPL
	00	0000	01	00	0	0	0	0
	00	0000	10	00	0	0	0	0
			00	00	0	0	0	0
			00	00	0	0	1	0
			00	00	1	0	0	0
			00	00	0	0	0	0
			00	00	0	1	0	0
			00	00	0	0	0	0
			00	00	0	0	1	1
			00	01	0	0	0	0
			00	10	0	0	0	0
			00	11	0	0	0	0

180 biți → 168 biți → 120 biți
pur orizontal. enc. by fct max. enc.

2.5.3. Formatul BA or CF

(a) Se cere ca at. când ∃ activ. în câmpul de cont, nu ∃ activ. în câmpul de adresă, și reciproc. 2 Excepții: → The End
→ de la adresa 6 (inc. COUNT + go to TEST 1)

↳ de regulă se folosește fie BA, fie CF cu excepțiile ↑

(b) Orice microprog. poate fi rescris a.î. să fie compus din instruct. de control (BA activ), fie instruct. funcționale (CF)

Fiind rescris astfel putem compacta câmpurile BA, CF, urmând ca rem-ref. câmpului să fie dedusă din tipul instrucțiunii

Scanned with CamScanner

Modificăm codul, pt. rândul unde avem c_4.

Separăm instrucțiunile: count ← count+1; . . . c_4
go to Test1;

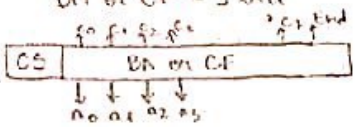
End: * //semnal — End — adresa 1100
go to End; — adresa 1101

CS <27> : cu aceeași codificare.

Vom ilustra metoda formatului multiplu, prin 2 exemple, unul fără codif. CF și celălalt care aplică la codif maximală a CF

(M1): BA = 4 biți
CF = 9 biți

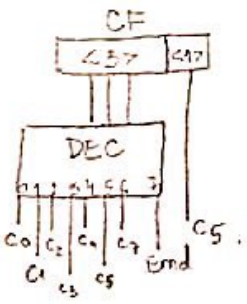
BA or CF = 9 biți



	instruct	CS	BA / CF	a_0 a_1 a_2 a_3 / c_0	d / c_1	d / c_2	d / c_3	d / c_4	d / c_5	d / c_6	d / c_7	d / End
	0000	00		1	0	0	0	0	0	0	0	0
	0001	00		0	1	0	0	0	0	0	0	0
	0010	01		0	1	0	0	0	0	0	0	0
	0011	00		0	0	1	0	0	0	0	0	0
	0100	00		0	0	0	1	0	0	0	0	0
	0101	10		1	0	0	0	0	0	0	0	0
	0110	00		0	0	0	0	1	0	0	0	0
	0111	11		0	0	1	0	0	0	0	0	0
Test1	1000	01		1	0	1	0	0	0	0	0	0
	1001	00		0	0	1	0	0	1	0	0	0
Output	1010	00		0	0	0	0	0	0	1	0	0
	1011	00		0	0	0	0	0	0	0	1	0
	1100	00		0	0	0	0	0	0	0	0	1
	1101	11		1	1	0	0	0	0	0	0	0

CM: 14 instruct. × 11 biți = 154 biți

(M2). Codificare maximală:



instruc-address	CS	BA / CF	a_3 a_2 a_1 a_0 / x_2 x_1 x_0 c_5
0000	00		0 0 0 0
0001	00		0 0 1 0
0010	01		0 1 0 0
0011	00		0 1 0 0
0100	00		0 1 1 0
0101	10		1 0 0 0
0110	00		1 0 0 0
0111	11		0 0 1 0
1000	01		1 0 1 0
1001	00		0 1 0 1
1010	00		1 0 1 0
1011	00		1 1 0 0
1100	00		1 1 1 0
1101	11		1 1 0 0

C.M: 14 instr × 6 biți = 84 biți

Scanned with CamScanner

CURS6

Cap. 3: Elemente de design ale procesoarelor
Seturi de instrucțiuni.

procesorul DLX : J. Hennesy, D. Petterson (cap. 2)

- stack
- accumulator
- register - memory
- register - register (load/store)

3 tipuri de instrucțiuni
- Immediate
- Register
- Jump

• Formatul **immediate** (imediat) I are 32 biți: opcode - 6 biți
sursă - 5 biți
dest. - 5 biți
restul - 16 biți - val. imediată

	31			0
I	opcode	rs	rd	immediate
	<6>	<5>	<5>	<16>

Folosit pentru:

ⓐ. instrucțiuni load/store: LW 40(R1), R3 <=> R3 ← Mem [40+R1]
SW R3(40), R1 <=> Mem [40+R1] ← R3

ⓑ. ALU immediate: ADDI R1, R3, #3 <=> R1 ← R3+3 (sau invers R1 ← R3+3)
SLL R1, R2, #2 <=> R2 ← R1 << 2

ⓒ. branch / jmp : BEQZ R4, #50 (offset) <=> if R4 == 0 then PC ← PC + offset (50)

• Formatul **register**:

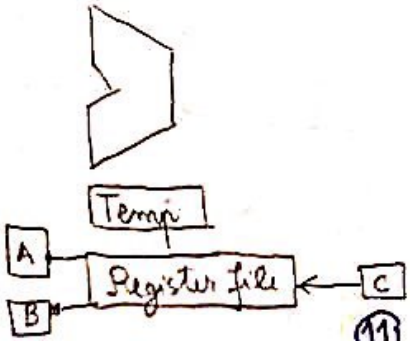
opcode	rs1	rs2	rd	funcție
<6>	<5>	<5>	<5>	<11>

ADD R1, R2, R3 <=> R1 ← R2 + R3

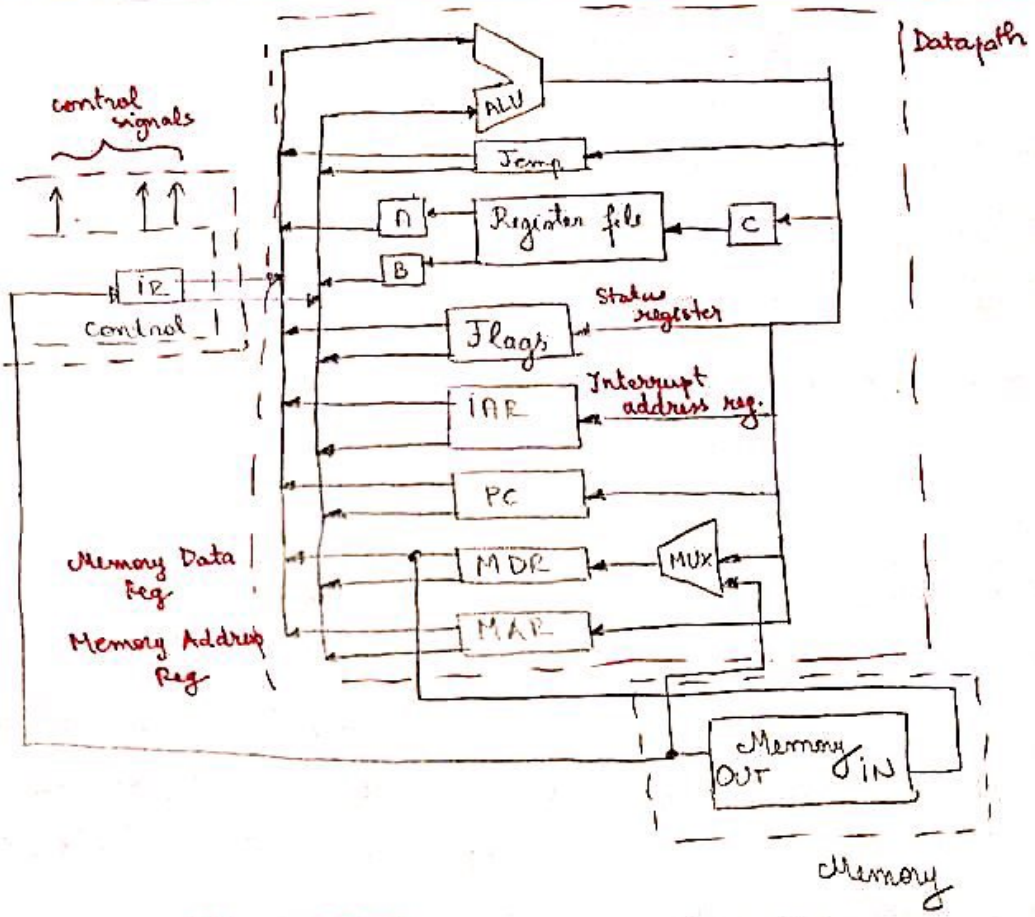
• Formatul **jump**:

opcode	offset cu semn
<6>	<26>

J offset <=> PC ← PC + offset



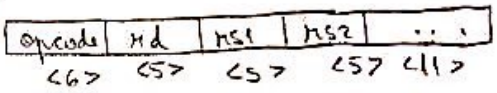
Scanned with CamScanner



DLX schematizat

(SB): CURS 7 Cap.4: Implementări ale setului de instruct.. Procesoare.

(4.1) Datapath



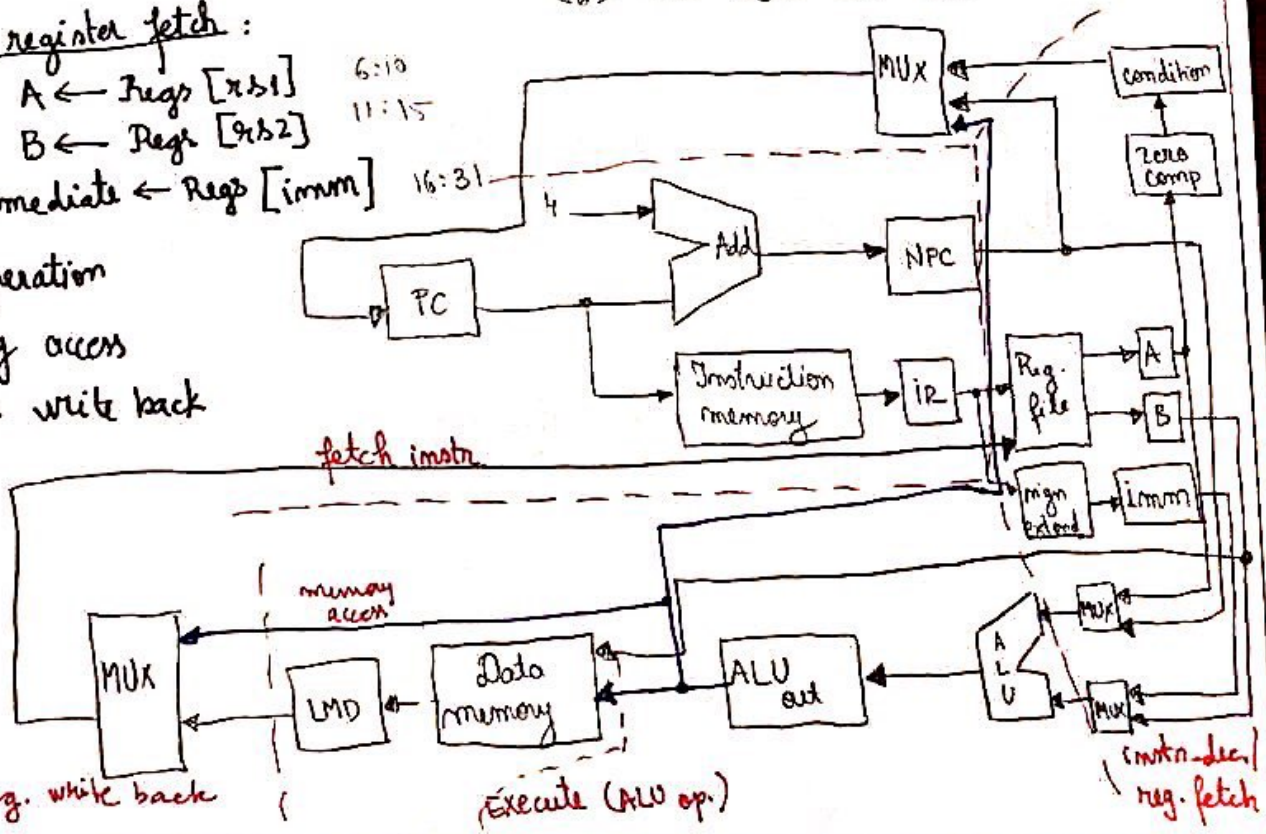
① Fetch IR ← Mem[PC]
PC ← PC+4

② Decode / register fetch:
A ← Regs[rs1] 6:10
B ← Regs[rs2] 11:15
immediate ← Regs[imm] 16:31

③ ALU operation

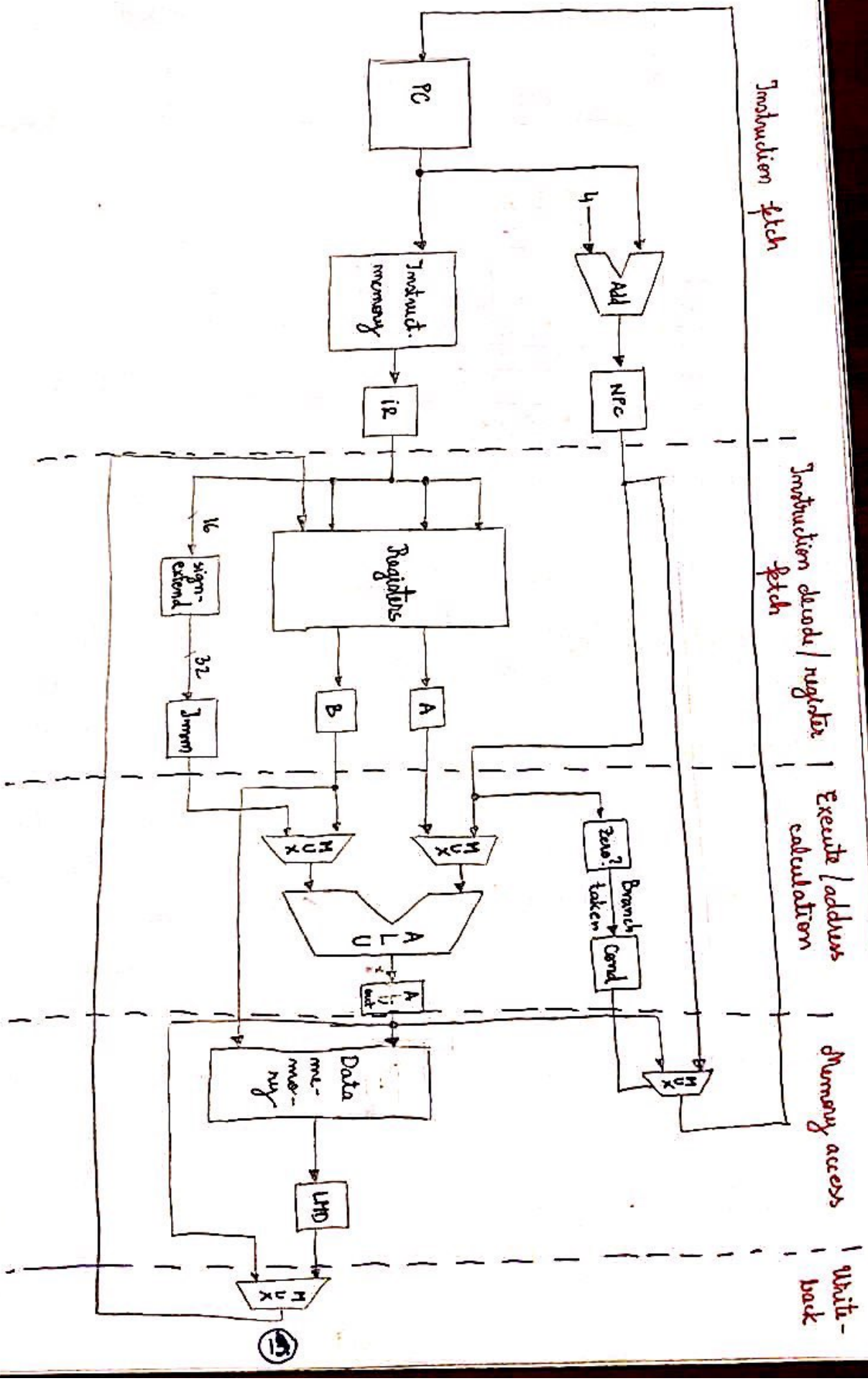
④. Memory access

⑤. Register write back



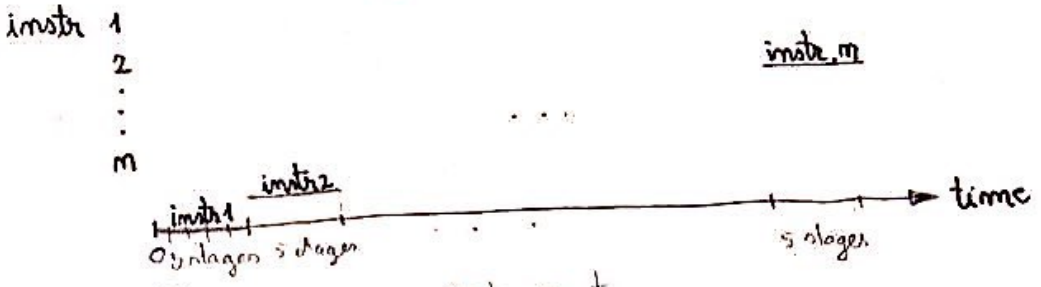
Scanned with CamScanner

Schema anterioară:

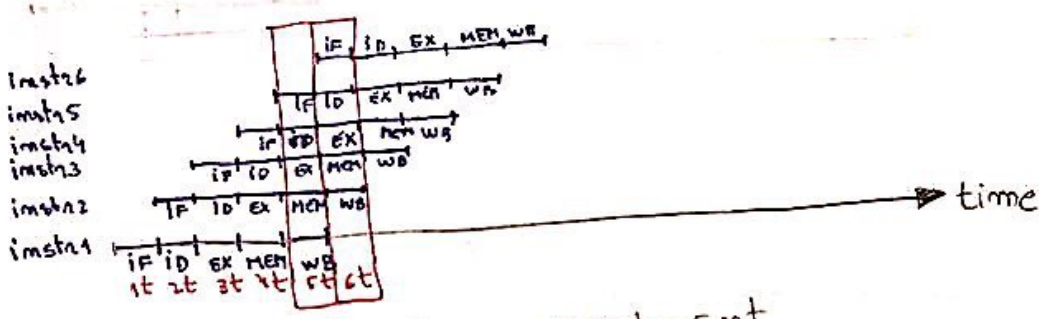


Scanned with CamScanner

1.2. Speeding up execution. A pipelined DLX datapath.



$$Time = m \times Time_{5\ stages} = m \times 5t = 5mt$$



$$time_{no\ pipeline} = m \times time_{5\ stages} = m \times 5t = 5mt$$

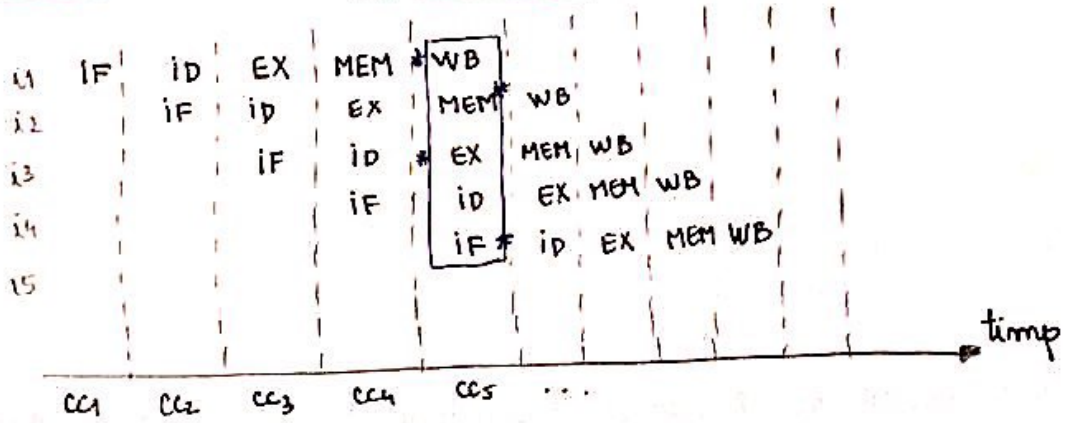
$$time_{with\ pipeline} = \underset{\substack{\uparrow \\ \text{prima instr.}}}{5t} + (m-1)t = 4t + mt = (m+4)t$$

$$speedup = \frac{time\ no\ pipeline}{time\ with\ pipeline} = \frac{5m\cancel{t}}{(m+4)\cancel{t}} = \frac{5m}{m+4} \longrightarrow 5$$

Scanned with CamScanner

CURS 8:

4.3. Hazarduri



non-pipeline: $\forall i$ 5 cc

program 5cc × m instr.

pipeline : program 4cc + m instr. × 1cc

$$\text{speedup} = \frac{5m}{4+m} \longrightarrow 5$$

Teoretical: speed-up = pipeline-depth

4.3.1. Hazardul structural

↳ se referă la conflictele care apar când d.p.d.v. hardware combinația de acces la resurse nu este suportată.

Sit. tipică este de suprapunere a inst. aritmetice, de regulă instr. care conțin întregi și instr. care conțin float, care pot duce la accesarea simultană a numătorului. O altă situație e atunci când în stagiul fetch se accesează memoria pt. a aduce instr., iar în stadiul MEM se accesează memoria pt. a transfera date. Memoria nu poate deservi 2 stadii în același timp ⇒

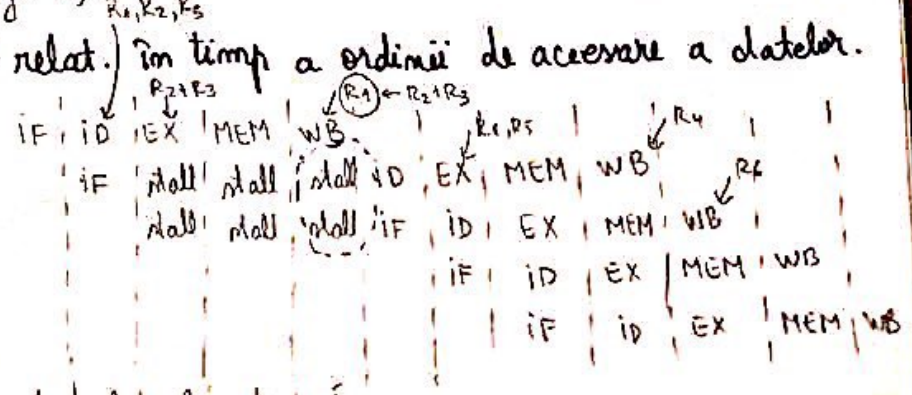
Hazard structural - eliminat din faza de concepție, asigurându-se posibilitatea execuției tuturor combinațiilor de instrucț.

4.3.2. Hazard de date (pag. 146)

↳ apare dat. schimbării relat. în timp a ordinii de accesare a datelor.

Exemplu:

ADD R_1, R_2, R_3
SUB R_4, R_1, R_5
AND R_6, R_1, R_7
OR R_8, R_1, R_9
XOR R_{10}, R_1, R_{11}



Scanned with CamScanner

În DECODE [scrierile devreme / citirile mai târziu] a unui ciclu de ceas.

⇓

pierdere mai mică

IF	ID	EX	MEM	WB						
	IF	stall	stall	ID	EX	MEM	WB			
		stall	stall	IF	ID	EX	MEM	WB		
					IF	ID	EX	MEM	WB	
						IF	ID	EX	MEM	WB

O tehnică anume pt. îmbunăt. hazardului de date este forwarding – se bazează pe remarca că val. de interes a prod. pe parcursul stadiului de ex. Dacă acea val. ar putea fi transferată la următ. instrucț., ar dispărea hazardul.

$R_1 \leftarrow R_2 + R_3$

$R_4 \leftarrow R_1 - R_5 \Leftrightarrow R_2 + R_3 - R_5$

i1 IF ID EX MEM WB

i2 IF ID EX MEM WB

Exemplu 2: LD R_1, 40(R_2) IF ID EX MEM WB

ADD R_5, R_1, R_4 IF ID stall EX

S10: 26.11.19.

CURS 9

forwarding / bypassing

LD R_1, 15(R_2) IF ID EX MEM WB

SUB R_3, R_1, R_2 IF ID stall EX MEM WB

ADD R_4, R_5, R_6 IF stall ID EX MEM WB

Pipeline interlock – detectează sit. de hazard și oprește pipeline-ul până când situația de hazard dispare.

①. a = b + c

LW R_b, b IF – ID – EX – MEM* – WB

LW R_c, c IF ID EX MEM* WB

ADD R_a, R_b, R_c IF ID stall EX MEM WB

SW a, R_a IF stall ID EX MEM

Scanned with CamScanner

② $a = b + c$
$d = e - f$

timp →

```
LW Rb, b
LW Rc, c
LW Re, e
ADD Ra, Rb, Rc
LW Rf, f
SW a, Ra
SUB Rd, Re, Rf
SW d, Rd
```

```
IF ID EX MEM WB ← Rb
   IF ID EX MEM WB ← Rc
      IF ID EX MEM WB ← Re
         IF ID EX MEM WB ← Ra
            IF ID EX MEM WB ← Rf
               IF ID EX MEM WB
                  IF ID EX MEM WB
                     IF ID EX MEM WB
```

metoda②:

```
LW Rb, b
LW Rc, c
LW Re, e
LW Rf, f
ADD Ra, Rb, Rc
SUB Rd, Re, Rf
SW a, Ra
SW d, Rd
```

```
IF ID EX MEM WB ← Rb
   IF ID EX MEM WB ← Rc
      IF ID EX MEM WB ← Re
         IF ID EX MEM WB ← Rf
            IF ID EX MEM WB ← Ra
               IF ID EX MEM WB ← Rd
                  IF ID EX MEM WB
                     IF ID EX MEM WB
```

⇒ mai rapid

Pipeline scheduling

Data hazards

- **RAW** (read after write) — i: write, j: read (j reads before i writes)
- **WAR** (write after read) — i: read, j: write (j writes before i reads)
- **WAW** (write after write) — i: write, j: write (j writes before j writes)
- **RAR** (read after read) — nu produce hazard

WAR — nu se poate întâmpla în procesorul DLX, deoarece toate citirile se fac devreme, iar scrierile târziu

WAW — nu se poate întâmpla — există un singur stadiu care scrie: WB.

4.3.3. Hazard de control

```
if x<y then go to Test;   DLX →
x ← x+1;
   ⋮
Test:                     EX
                          if x<y then PC ← value
                                 else PC ← PC+4
```


Scanned with CamScanner

if conditie then —
else —

DLX ⇒ BNEZ / BEQZ } if $R_x = 0$ PC ← new PC
else PC ← PC + 4

if cond then
br. successor

branches → taken, branch target
→ not taken, branch successor, fall through

BEQZ R1, label
— — —
— — —

Label :

branch instr. BEQZ R1, Label
branch succes : ADD R2, R3, R4
succ+1
⋮

zero cond NPC

IF ID EX MEM WB
IF stall stall IF ID ...
3 cycles stall

30% branches ⇒ pipeline speedup −50%

10000

- IF : IR ← Mem [PC]
  NPC { branch instr.: ALU (if zero test)
  { no branch instr.: PC+4

ID : A ← Regs [IR 6÷10]
B ← Regs [IR 11÷15]
IMM ← IR_{16}^{16} ## $IR_{16 \div 31}$

EX ALU out ← A func B
zero test

MEM LMD ← Mem [ALU out]
NPC ← { branch : ALU out (if zero test)
no branch instr. : PC

WB : Regs [IR 16÷20] / Regs [IR 11÷15] ← ALU out / LMD

pipeline freezing / stopping / flushing

− zero test
− NPC } earlier (ne propune)

IF − ID − EX − MEM − WB branch instr.
IF − IF − ID − EX ... branch succ / br. target
1 cycle stall

BEQZ R1, label
ADD R2, R3, R4

Scanned with CamScanner

4 moduri de tratare a instrucț. de salt

1. pipeline freezing – cea mai simplă
   – penalty 3 cc (fix)

2. predict not taken
   - branch untaken IF - ID - EX - MEM - WB
     successor IF - ID - EX - MEM - WB
   - branch taken IF - ID - EX - MEM - WB
     br. succ. / br. target IF - IF - ID - EX...
     1 cycle stall

3. predict as taken (pt. DLX nu are avantaje)

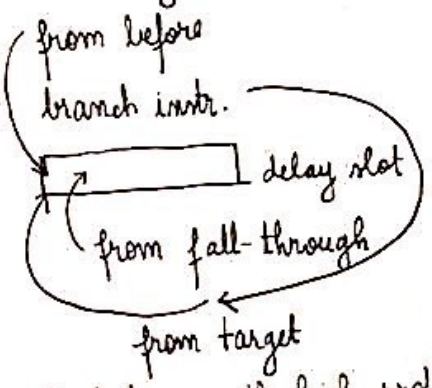
4. branch delay slot

CURS 11

Treating branches

? cycle stall ≃ other instr. ?

- 1 branch delay slot



from before

```
ADD R1, R1, R3
if R2=0 then
[ ]
if R2=0 then
[ADD R1, R1, R3]
```

from target

```
SUB R4, R5, R6
ADD R1, R2, R3
if R1=0 then
[ ]
SUB R4, R5, R6
ADD R1, R2, R3
if R1=0 then
[SUB R4, R5, R6]
```

branch taken with high prob.

from fall-through

```
ADD R1, R2, R3
if R1=0 then
[ ]
OR R7, R8, R9
SUB R4, R5, R6
↓
ADD R1, R2, R3
if R1=0 then
[OR R7, R8, R9]
SUB R4, R5, R6
```

Concluzie: from target / fall-through – e cu probleme
⇓
se bazează pe abilitatea de a „prezice" (predict)

Scanned with CamScanner

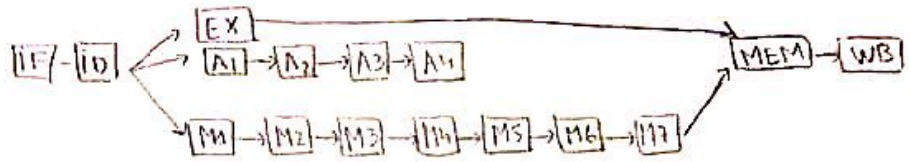
4.4 : Extinderea pipeline pentru operații multicicle

IF ID EX MEM WB

1cc — load / store, branch, ALU integer

4cc FP addition — 4cc int multiplication

7cc FP multiplication



Issues: 5cc, 8cc, 11cc instruction length ⇒ multiple WB/cc ⇒ Structural hazard

- instructions no longer in order ⇒ Data hazards WAR, RAW, WAW
  ↳ interrupts

```
LD   F4, 10(R3)        IF - ID - EX - MEM - WB
MULT D F0, F4, F6      IF - ID - stall - M1 - M2 - M3 - M4 - M5 - M6 - M7 - MEM - WB
ADDD F2, F0, F8        IF - stall - ID - stall - stall . . . . . A1 - A2 ...
SD   10(R2), F2        IF - ⊙ - ⊙                                ID - EX
                                                                 A3 - A4 - MEM - WB
                                                                 stall - stall - (MEM)
```

teoretic hazard de date structural, dar ADDD nu folosește MEM!!!

2 soluții: - supraveghere porturi de scriere și întârzierea instruct. - dep
- întârziere instr. conflictuale

4.5 Loop unrolling

```
for (i = 1000; i > 0; i--)   x[i] = x[i] + S
Loop: LD   F0, 0(R1)     ; F0 ← x[i], R1 contains i
      ADD  F4, F0, F2    F2 contains S
      SD   0(R1), F4
      SUBI R1, R1, #8
      BNEZ R1, Loop
```

```
① IF ID EX MEM WB
② IF ID stall A1 A2 A3 A4 MEM WB                    ⇒ 13 CC
③ IF stall ID EX ⊙ ⊙ ⊙ MEM WB
④ IF ID ⊙ ⊙ ⊙ EX MEM WB
⑤ IF ⊙ ⊙ ⊙ ID EX MEM WB
```

```
Loop LD F0, 0(R1)      IF ID EX MEM WB
     SUBI R1, R1, #8   IF ID EX MEM WB
     ADDD F4, F0, F2   IF ID A1 A2 A3 A4 MEM WB          ⇒ 11 CC
     BNEZ R1, Loop     IF ID EX MEM WB
     SD 8(R1), F4      IF ID EX ⊙ ⊙ MEM WB
```

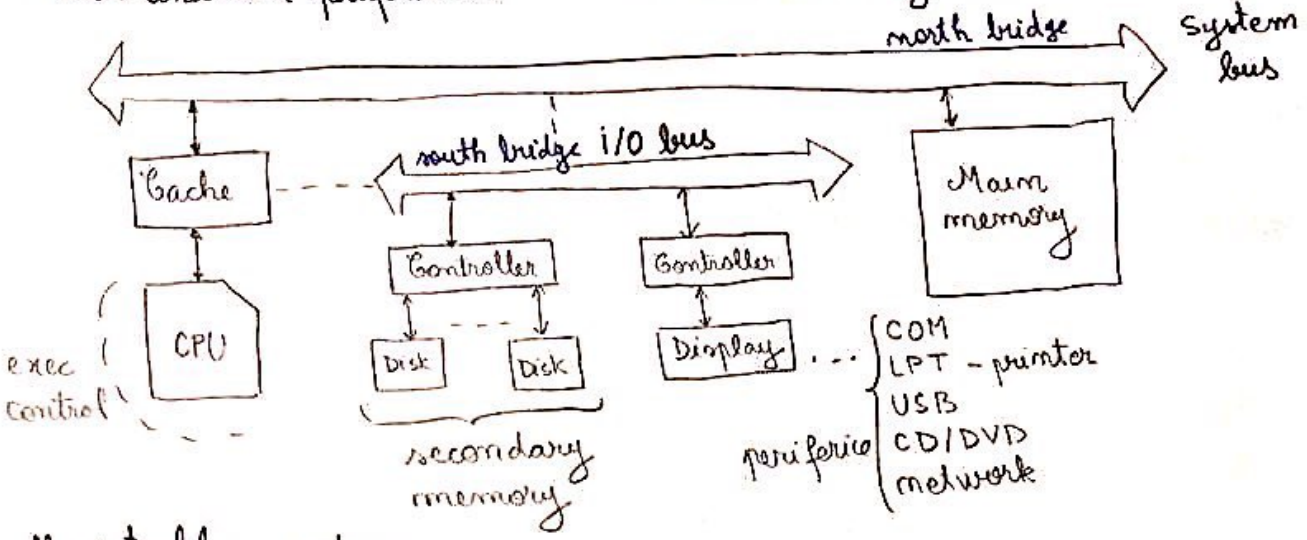
pipeline fall → identify parrallel instr. (non-conflictual)

Scanned with CamScanner

CURS 12

CAP. 5 : Sistemul de intrare/ieșire (I/O)

- interconectarea perifericelor
- accesul la magistrale



Magistralele:
- viteza
- lățime (width)
- bus master

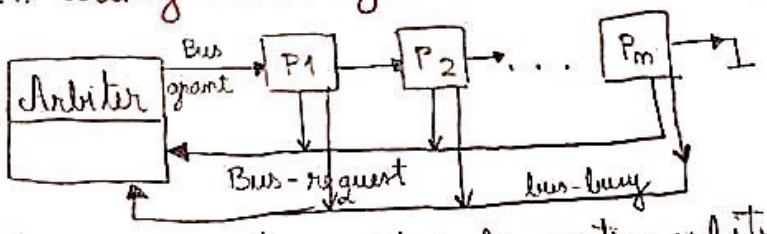
transaction → split transaction
- request : adresa + semnale control | bus master
- reply : sunt livrate datele (data delivery)

5.1. Arbitrarea pe magistrale

↳ este necesară pt. mai multe dispozitive care pot fi bus master

1. daisy-chaining
2. polling
3. independent requesting

5.1.1. Daisy chaining



↳ sunt transparente periferice pentru arbitru

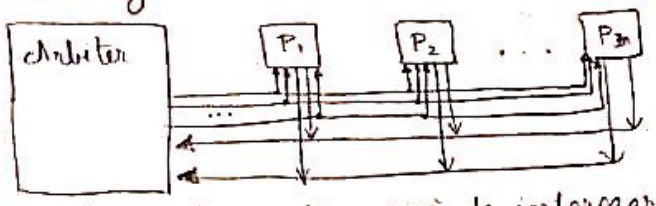
↳ primul care dorește acces la magistrală devine opac și bus busy devine activ → câtă vreme e activ arbitrul nu va lua alte request-uri

PROS: cheap

CONS: slow, not fair

Scanned with CamScanner

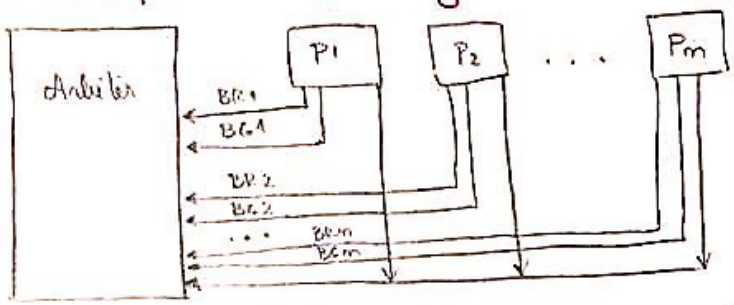
5.1.2. Polling



] polling lines [$\log_2 m$]

PROS: fair; ordinea dinamică de interogare
CONS: pricy, să existe linii pt. toate periferiile

5.1.3. Independent requesting



PROS: fair, fast
CONS: price complexity

5.2. Interfața cu CPU

– i/o bus conectat cu System bus și accesul la cache și memoria principală sunt cele ce trebuie efectuate.

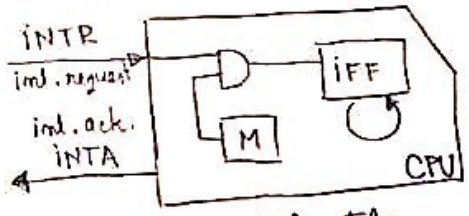
memory – mapped i/O: transf. de date între CPU și periferice se fac indirect prin int. unor zone de mem. dedicate (prin instr. standard de transf. cu mem.)

i/O – mapped: instrucțiuni dedicated opcodes lines
(→ transferul se face prin instr. dedicate (între CPU periferic) – astfel de inst. accesează linii dedicate pt. transfer

Indif. de metoda de adresare, fiecare periferic conține regiștrii pt. date și pt. informație de stare.

Op. cu periferice nu sunt desf. prin instr. singulare (transfer de mai multe date) — 1 cuvânt x m ori
— m cuvinte într-o singură operație

Exemplu: tipărirea unei imprimante: CPU → reg. date imprimantă
→ interoghează starea
⇓
Întreruperi (mai eficient)
– nu mai interog. starea
– imprimanta când încheie tipărirea informează CPU de acest fapt cu semnal de întrerupere. ⇒ CPU sare la o rutină specială – tratare întrerupere prin care identifică sursa interup. și ia decizia (tipărirea altui caracter)



IFF – Interrupt Flip Flop
M – masking — maskable interrupts
— non-maskable int.

Întreruperile – principalul mijloc prin care disp. i/O lente cer servicii din CPU

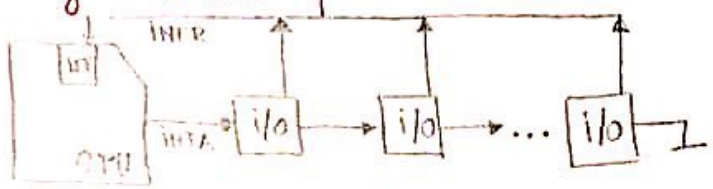
Scanned with CamScanner

1° CPU să cunoască sursa întreruperii.
2° Cereri simultane de întreruperi ⟶ arbitrare
3° Adresa rutinei de tratare a întreruperii

Tratarea se face într-o manieră asemănătoare rulării ... [...]... se execută subrutina, se revine printr-o instrucțiune specială IRET (return from interrupt) — are rolul nu doar de a reface starea CPU, ci și starea mecanismului de întreruperi (CPU devine din nou disponibil pt. întreruperi)

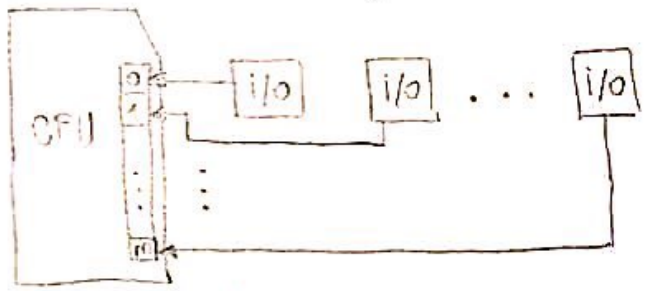
5.3 Selecția întreruperii

5.3.1. Single line interrupt



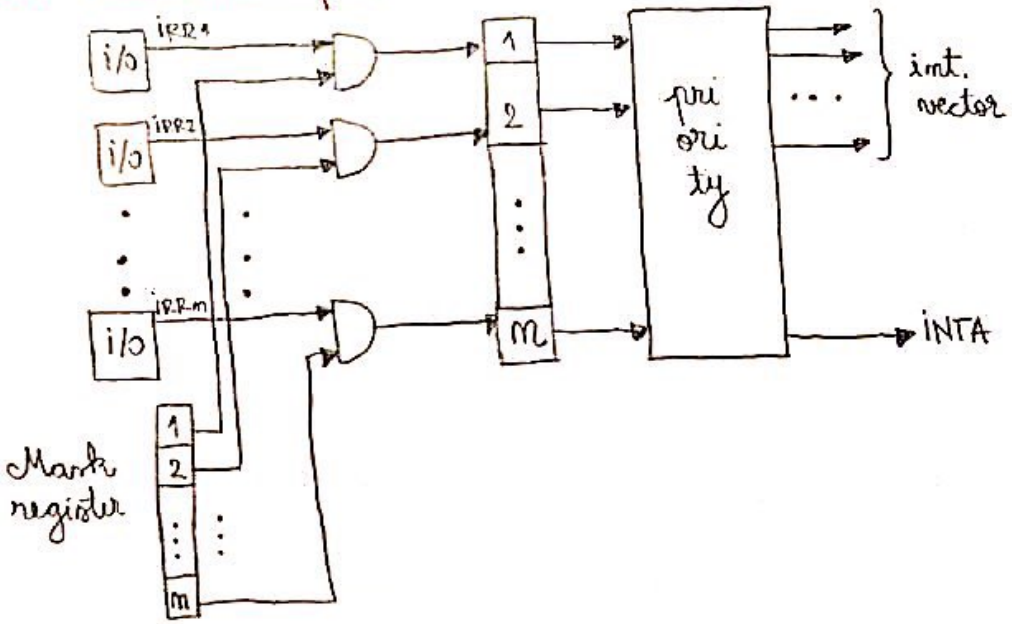
INTA – dat în manieră daisy-chaining.

5.3.2. Multiple line interrupt



CPU interoghează registrul.

5.3.3. Vector interrupts



Scanned with CamScanner